

vvEntra Venture Intelligence Platform

Frontend Technology Stack

1. Summary

This document describes the technology proposed for the web frontend of the vvEntra platform and the plan to deliver it. The frontend is a high-performance web application built with Next.js 16, React 19, and TypeScript that connects to the Python/FastAPI backend API over a versioned REST interface. It is designed to be fast, responsive, and secure, delivering an institutional-grade interface for verified micro-SaaS acquisitions, financial audits, and clean-room transfers.

- **Unified, component-driven design.** Next.js App Router and a tailored design system using Tailwind CSS and shadcn/ui primitives keep the MVP fast to load, elegant to navigate, and straightforward to maintain.
- **Type safety end-to-end.** TypeScript across all pages, components, and API client layers eliminates runtime schema mismatches with the backend.
- **Member security and confidentiality built in:** strict client-side validation, short-lived tokens in secure cookies, sanitization of user data, and clear role isolation between investors and creators.
- **Decoupled monorepo architecture.** The frontend lives in its own directory, enabling independent deployment and conflict-free parallel development alongside the backend team.

2. Architecture overview

- **Next.js App Router** renders pages at the edge, serving static public content instantly while delivering dynamic member experiences for authenticated workflows.
- **Frontend communicates with the FastAPI backend** over HTTPS through versioned REST endpoints. The interactive OpenAPI schema serves as the binding contract between both developers.
- **TanStack Query (React Query)** handles server state: automated caching, background polling, request deduplication, and optimistic updates for live market data.
- **React Hook Form with Zod** manages client form state, running real-time constraint validation and formatting before any request touches the network.
- **Direct S3/R2 uploads.** Large pitch decks and verification assets are uploaded directly to cloud storage via secure, time-limited presigned URLs generated by the backend API.
- **Role-gated route architecture** enforces strict separation between Investor exploration (deal teasers, confidentiality requests) and Architect management (listing drafts, revenue audits).

3. Technology stack

Layer	Technology	Purpose and rationale
Language and tooling	TypeScript 5, Node.js 20+, npm	Provides compile-time type safety across all components, API schemas, and utility functions. Catches data errors early and streamlines developer collaboration.
Core framework	Next.js 16 (App Router, Turbopack)	Leading React framework offering sub-second builds, hybrid server/client rendering, automatic font/image optimization, and zero-config edge routing.
UI library	React 19 (Server & Client Components)	Modern concurrent React primitives, robust component state, and declarative UI hierarchy for responsive deal portals.
Styling and design tokens	Tailwind CSS v4, CSS Variables	High-speed utility-first engine with custom design tokens for role-based themes (Investor orange vs. Architect green) and light/dark modes.
Component primitives	shadcn/ui foundation, Radix UI, Lucide Icons	Accessible, unstyled UI primitives tailored to vvEntra's bespoke luxury editorial identity without the bloat of heavy commercial component libraries.
Data fetching and cache	TanStack Query v5 (React Query)	Declarative server-state management. Caches queries, manages background sync, and handles automatic retries for real-time market data.
Form handling and validation	React Hook Form, Zod	High-performance form state with isolated re-renders. Zod schemas validate email formats, password security constraints, and listing forms.
Data visualization	Recharts (SVG-based)	Interactive, responsive financial charts: MRR curves, valuation multiples, revenue distributions, and escrow milestones.
Authentication client	Google OAuth, LinkedIn OAuth, Secure cookie client	Handles dual SSO integrations and email auth flows. Works in harmony with backend rotating refresh tokens and httpOnly cookies.

Layer	Technology	Purpose and rationale
Document preview	PDF.js / Canvas viewer, Presigned S3 client	Renders watermark-protected previews of pitch decks and financial disclosures directly in-browser without local file retention.
Quality and E2E testing	Playwright (Chromium, Firefox, WebKit)	Automated end-to-end test suite testing key user journeys: authentication, role switching, listing creation, and marketplace filtering.
Hosting and edge delivery	Vercel (Global Edge Network)	Zero-configuration CI/CD directly connected to GitHub. Automatically deploys preview environments on every branch push.
Code quality and linting	ESLint 9, Prettier	Enforces uniform code standards, TypeScript strictness, and prevents common React anti-patterns before commits.

4. Why this approach

Next.js 16 and TypeScript. Enables a single frontend developer to move rapidly while providing institutional-level stability, excellent SEO discovery for public listings, and native SSR speed.

TanStack Query with React Hook Form and Zod. Cleanly isolates server data from UI state. Prevents malformed payloads from ever hitting the backend and provides instant, zero-latency validation feedback to users.

Tailwind CSS and custom primitives. Heavy off-the-shelf component suites look generic and add stylesheet bloat. Building on lightweight primitives ensures vvEntra's distinctive editorial aesthetic (Instrument Serif typography, warm paper palettes, refined micro-interactions) is preserved.

Playwright for end-to-end verification. Financial transactions and cap-table workflows require complete reliability. Automated browser tests verify complete journeys on every build.

5. Security and data protection

- **Zero secret credentials in client code.** All sensitive API keys and secrets reside strictly in environment variables; only client-safe keys prefixed with NEXT_PUBLIC_ are exposed.
- **Secure token lifecycle.** Operates alongside the backend's 15-minute access tokens and rotating refresh tokens using httpOnly, SameSite, and Secure cookies to prevent token theft.
- **Role-based route protection.** Strict client guards and middleware ensure Investors cannot access Architect publishing portals, and Creators cannot access confidential buyer lists.
- **XSS prevention and input sanitization.** All user-supplied text and listing descriptions are sanitized before rendering.
- **Confidential data handling.** Sensitive financial teasers and cap tables are loaded on-demand into memory and never stored in unencrypted browser storage (localStorage).

6. Risks and mitigations

Risk	Mitigation
API schema drift between teams	FastAPI generates interactive OpenAPI specifications; frontend auto-generates or verifies TypeScript types against the schema.
Large asset upload latency (Pitch Decks)	Client uploads directly to S3/R2 storage via presigned URLs with chunking and progress indicators, bypassing the web server.
Complex financial chart performance	Use lightweight SVG paths in Recharts with debounced resize observers and container virtualization.
Token expiry during long listing drafts	Silent background token refresh via API interceptor, combined with client-side draft auto-save.
Cross-browser and mobile inconsistencies	Playwright automated tests run across Chromium, Firefox, and WebKit on mobile, tablet, and desktop viewports.
Dependence on a single frontend developer	Strict TypeScript interfaces, modular component architecture, and comprehensive Playwright tests ensure easy onboarding and review.
Handling sensitive investor information	Confidential documents render behind authenticated access controls with watermarking, avoiding client-side caching.